基于 Pytorch 的深度学习实践（三）

YYFL

March 2026

1 卷积神经网络（CNN）

卷积神经网络是专为处理**网格结构数据（如图像）**设计的深度学习模型，核心通过「卷积层 + 池化层」替代全连接层，解决全连接层参数爆炸、缺乏空间信息利用的问题，核心优势：

1. **局部感受野**：每个神经元仅关注输入的局部区域，模拟人类视觉系统的局部感知特性；
2. **权值共享**：同一卷积核在整个输入上重复使用，大幅减少参数数量；
3. **空间不变性**：通过池化/卷积的平移不变性，对图像的缩放、平移、旋转等变形具有鲁棒性。

2 CNN 核心组件

2.1 卷积层（Convolution Layer）

2.1.1 核心概念

- **卷积核（Kernel/Filter）**：小尺寸的权重矩阵（如 3×3、5×5），滑动遍历输入特征图，通过点积计算输出；
- **步长（Stride）**：卷积核每次滑动的像素数（默认 1）；
- **填充（Padding）**：在输入特征图边缘补 0，保持输出尺寸与输入一致（Same Padding）或缩小（Valid Padding）；
- **输入/输出尺寸计算**：设输入尺寸 $H \times W \times C_{in}$，卷积核尺寸 $K \times K$，步长 S，填充 P，则输出尺寸：

$$H_{out} = \lfloor \frac{H + 2P - K}{S} + 1 \rfloor, \quad W_{out} = \lfloor \frac{W + 2P - K}{S} + 1 \rfloor, \quad C_{out} = \text{卷积核数量}$$

2.1.2 卷积计算示例

输入特征图（单通道，3×3）：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

卷积核（3×3，步长 1，Same Padding）：

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

卷积输出（点积求和）：

$$1\times1+2\times0+3\times(-1)+4\times2+5\times0+6\times(-2)+7\times1+8\times0+9\times(-1)=-4$$

2.2 激活函数层（Activation Layer）

卷积输出后需加非线性激活，引入非线性表达能力：

激活函数	公式	优势	适用场景
ReLU	$\max(0,x)$	缓解梯度消失，计算快	隐藏层首选
Leaky ReLU	$\max(\alpha x,x)(\alpha=0.01)$	解决 ReLU 死亡神经元问题	深层 CNN
Sigmoid	$\frac{1}{1+e^{-x}}$	输出 0 1，适合二分类	输出层（少用）
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	输出概率分布	多分类输出层

表 1: CNN 常用激活函数

2.3 池化层（Pooling Layer）

对特征图降维，保留关键信息，减少计算量：

- **最大池化（Max Pooling）**：取局部区域最大值（保留纹理/边缘特征）；
- **平均池化（Average Pooling）**：取局部区域平均值（保留整体特征）；
- 常用尺寸：2×2，步长 2（输出尺寸减半）。

2.4 全连接层（FC Layer）

将池化层输出的二维特征图展平为一维向量，通过全连接层完成分类/回归任务。

3 经典 CNN 架构

架构	核心创新	适用场景
LeNet-5	首个实用 CNN，卷积 + 池化 + 全连接	手写数字识别（MNIST）
AlexNet	ReLU 激活 +Dropout+GPU 训练	图像分类（ImageNet）
VGG-16/19	堆叠小尺寸卷积核（3×3），加深网络	特征提取、迁移学习
ResNet	残差连接（Skip Connection）	深层网络（100+ 层），缓解梯度消失
MobileNet	深度可分离卷积	移动端/轻量化场景

表 2: 经典 CNN 架构对比

4 CNN 反向传播

CNN 反向传播需分别计算「卷积核权重梯度」「偏置梯度」「输入特征图梯度」，核心仍基于链式法则：

设卷积层前向计算：$Y = X * K + b$（$*$ 为卷积操作），损失 L，则：

1. 偏置梯度：$\frac{\partial L}{\partial b} = \sum_{i,j} \frac{\partial L}{\partial Y_{i,j}}$（输出梯度求和）；
2. 卷积核梯度：$\frac{\partial L}{\partial K} = X * \frac{\partial L}{\partial Y}$（输入与输出梯度做「全卷积」）；
3. 输入梯度：$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} * K_{\text{rot180}}$（输出梯度与旋转 180° 的卷积核卷积）。

池化层反向传播：

- 最大池化：仅将梯度传递给前向计算时的最大值位置，其余位置梯度为 0；
- 平均池化：将梯度平均分配到前向计算的对应区域所有位置。

5 CNN 代码实现（PyTorch）

5.1 基础 CNN 实现（MNIST 分类）

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms

# 数据预处理
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))  # MNIST均值/标准差
])

# 加载数据集
train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)

# 定义CNN模型
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # 卷积层1：输入1通道，输出16通道，3×3卷积，Same Padding
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        # 池化层：2×2，步长2
        self.pool = nn.MaxPool2d(2, 2)
        # 卷积层2：输入16通道，输出32通道
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
```

```python
        # 全连接层1: 32*7*7 → 128 (池化后尺寸: 28/2/2=7)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        # 全连接层2: 128 → 10 (10分类)
        self.fc2 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(0.5)  # 防止过拟合

    def forward(self, x):
        # 前向传播: conv1 → relu → pool → conv2 → relu → pool → flatten → fc1 → dropout → fc2
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 32 * 7 * 7)  # 展平
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.fc2(x)
        return x

# 初始化模型、损失函数、优化器
model = SimpleCNN()
criterion = nn.CrossEntropyLoss()  # 交叉熵损失 (多分类)
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练模型
def train(model, loader, criterion, optimizer, epochs=5):
    model.train()  # 训练模式
    for epoch in range(epochs):
        running_loss = 0.0
        for batch_idx, (data, target) in enumerate(loader):
            # 前向传播
            output = model(data)
            loss = criterion(output, target)

            # 反向传播+优化
            optimizer.zero_grad()  # 梯度清零
            loss.backward()        # 反向传播
            optimizer.step()       # 参数更新

            running_loss += loss.item()
            if batch_idx % 100 == 99:
                print(f'Epoch {epoch+1}, Batch {batch_idx+1}, Loss: {running_loss/100:.4f}')
                running_loss = 0.0

# 启动训练
train(model, train_loader, criterion, optimizer)
```

6 CNN 典型应用场景

1. **图像分类**：ImageNet、MNIST、CIFAR-10/100；

2. **目标检测**：YOLO、Faster R-CNN、SSD；

3. **图像分割**：U-Net、Mask R-CNN；

4. **人脸识别**：FaceNet、ArcFace；

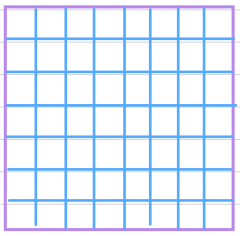
5. **医学影像分析**：CT/MRI 图像病灶检测。

A 手写笔记

A.1 笔记图片

深入 CNN　局部相关性　平移不变性

input：224×224×3 ≈15w　假设 Hidden Layer1 有1000个神经元，参数量达 w×1000 = 1.5亿 参数爆炸



三大核心组件

1. 卷积层 Conv layer　　优势 ① 局部感受野

卷积核 kernel 滑动窗口，加权求和　　② 权值共享，减少参数量

对于 RGB 图像来说，3D Kernel (H, W, C) 是在平面上移动，

不过同时与 R. G. B 三个通道加权求和，产生一个通道，特征图

1个 kernel 对应 1个 输出通道

关键超参数 1° kernel-size 卷积核大小 2° stride（滑动步长）控制输出尺寸 3° Padding 边缘填充0（保持图像尺寸不变）

2. 激活函数 ReLU( )：max{0, x}

3. 池化层 Pooling Layer　　减少参数量

{ 最大池化 Max Pooling / 平均池化 Average Pooling } → { 7 / 4 }